

챕터 10. PDF 이미지까지 잡아라. Vision LLM과 RAG 평가

이번 챕터가 끝나면

- 스캔 PDF / 이미지 문서를 OCR과 Vision LLM으로 읽어 벡터 DB에 넣습니다
- 하이브리드 파서로 이미지가 있는 페이지는 Vision LLM, 텍스트만 있는 페이지는 pypdf로 자동 분기합니다
- RAG 평가 프레임워크로 Precision@k, Recall, Hallucination Rate를 숫자로 측정합니다
- 느낌이 아니라 숫자로 품질을 판단하는 습관을 가집니다

준비하기. 실습 시작 전 한 번만 설정

1. 실습 폴더 이동

터미널 폴더 이동

```
cd rag-start/ex10
```

파일 구조는 다음과 같습니다.

ex10 디렉토리

```
ex10/
├─ requirements.txt          # [참고] easyocr · PyMuPDF · chromadb 등
├─ data/
│   └─ docs/                # [참고] 원본 PDF (텍스트 + 스캔본)
│   └─ test_questions.json  # [참고] 평가 셋 (질문·정답·근거 문서)
│   └─ chroma_db/           # 런타임 생성 (평가 프레임워크 벡터 DB)
├─ tuning/
│   └─ step1_document_parser/ # [실습] OCR / Vision LLM 파서 비교
│       └─ __main__.py
│       └─ ocr.py            # [실습] EasyOCR 파서
│       └─ vision.py         # [실습] Vision LLM 파서
```

```

├── __main__.py
├── hybrid_parser.py      # [실습] 하이브리드 파서
├── display.py
└── step3_eval_framework/ # [실습] Precision@k · Recall · 환각률 + A/B/
C/D 조합 평가
    ├── __main__.py
    ├── metrics.py        # [실습] Precision@k · Recall@k · 환각률
    ├── strategies.py     # [참고] A/B/C/D 4 조합 정의
    ├── pipelines.py      # [참고] 파싱·청킹·쿼리·검색·리랭크 부품
    ├── evaluator.py      # [참고] run_evaluation(strategy)
    └── display.py

```

챗터 7의 FastAPI 서버·채팅 UI·에이전트 레이어는 **챗터 11** · [ex11/](#) **레포**에서 완성 형태로 사용합니다.
 챗터 10의 ex10은 튜닝 실험만 다룹니다.

2. 실습 환경 구축

터미널 환경 구성. macOS / Linux

```

cd ex10
python3.12 -m venv .venv
source .venv/bin/activate
cp .env.example .env
ollama pull qwen2.5vl:7b
pip install -r requirements.txt

```

터미널 환경 구성. Windows

```

cd ex10
py -3.12 -m venv .venv
.venv\Scripts\activate
copy .env.example .env
ollama pull qwen2.5vl:7b
pip install -r requirements.txt

```

Vision LLM, 어떤 모델로 실습을 돌릴지 먼저 정하세요

EasyOCR은 `Reader(["ko", "en"])` 처럼 언어 코드만 지정하면 한국어+영어를 함께 인식하니 별다른 선택지가 없습니다. 갈림길은 **Ollama Vision 모델** 쪽입니다. 파라미터 수가 성능과 요구 사양을 동시에 결정하고, **3B 이하**는 노트북 CPU에서도 돌지만 한글·표 인식이 불안정합니다. **7~8B**가 로컬 실무의 현실적 기준선이고, **13B 이상**은 품질이 더 좋지만 GPU가 사실상 필수입니다.

모델	VRAM/RAM	한국어 품질	권장 환경
<code>qwen2.5vl:7b</code>	8GB 이상	중음	RAM 8~16GB 노트북 (기본 권장)
<code>minicpm-v:latest</code>	8GB 이상	괜찮음 (환각 있음)	노트북, 빠른 실습용
<code>llama3.2-vision:11b</code>	16GB 이상	중음	RAM 16GB 이상 데스크톱 (고품질)

- 양자화(Quantization)** : Ollama 태그에 보이는 `q4_0` · `q5_K_M` 는 모델 가중치를 4~5비트로 압축했다는 뜻입니다. 용량과 RAM 사용량을 30~50% 줄여 주는 대신 정확도가 살짝 내려갑니다. 기본 태그(`:latest`)는 대개 품질과 크기의 절충점을 골라 둔 버전이라 실습에서는 그대로 써도 충분합니다.
- 고르는 순서**: (1) 내 머신 RAM·VRAM 확인 → (2) 그 안에 들어가는 후보 1~2개 선정 → (3) 사내 문서 5~10장으로 품질 비교. 벤치마크 점수보다 **우리 문서에서 잘 읽는지가** 최종 기준입니다.
- 상용 API를 쓸 수 있는 경우**: 인사 기록·계약서처럼 **외부 유출이 금지된 사내 문서**는 품질이 좋아도 API를 선택할 수 없습니다. 공개 자료나 외부 공문처럼 유출 제약이 없다면, **GPT-4V가 아니라** `gpt-4o-mini` 부터 검토하세요. Vision 입력을 그대로 지원하면서 **GPT-4o의 약 1/10 요금**으로, 표·스캔본 정도는 품질 차이를 체감하기 어렵습니다.

3. 사용할 라이브러리

패키지	역할
<code>easyocr</code>	한국어+영어 OCR (EasyOCR)
<code>PyMuPDF (import fitz)</code>	PDF 페이지를 이미지로 변환

Pillow

이미지 전·후처리

langchain-ollama

Vision LLM 호출 (멀티모달 메시지)

pypdf

텍스트 레이어 감지·추출

4. 실습 순서

1. `python -m tuning.step1_document_parser`. OCR vs Vision LLM 비교
2. `python -m tuning.step2_hybrid_parser`. 이미지 감지 후 자동 분기
3. `python -m tuning.step3_eval_framework`. Precision@k, Hallucination Rate 측정

10.1 스캔 PDF: 텍스트가 없다



그림 10-1. 스캔 PDF 문제 해결. OCR + Vision LLM으로 이미지도 읽고, 숫자로 품질을 측정합니다

챗터 9까지 검색과 쿼리, 모든 축을 다듬었습니다. 금요일 오후, 키보드 소리만 또렷또렷 울리는 사무실에서 오픈이가 모니터를 정리하고 있을 때 팀장이 다가왔습니다.

팀장 : "이것도 넣어 줘요. 정보보안서약서."

A4 용지를 스캔한 듯한 PDF 한 장. 오픈이가 기존 파이프라인에 넣었습니다. pypdf 결과, 빈 문자열. 한 글자도 안 나왔습니다.

파일을 열어 봤습니다. 스캔본이었습니다. 종이를 스캐너에 올려 찍은 PDF라 페이지 전체가 하나의 큰 이미지. 텍스트 레이어가 없으니 pypdf가 꺼낼 글자 자체가 없었습니다.

사람 눈에는 선명하게 보이는데, 사서한테는 백지나 마찬가지잖아.

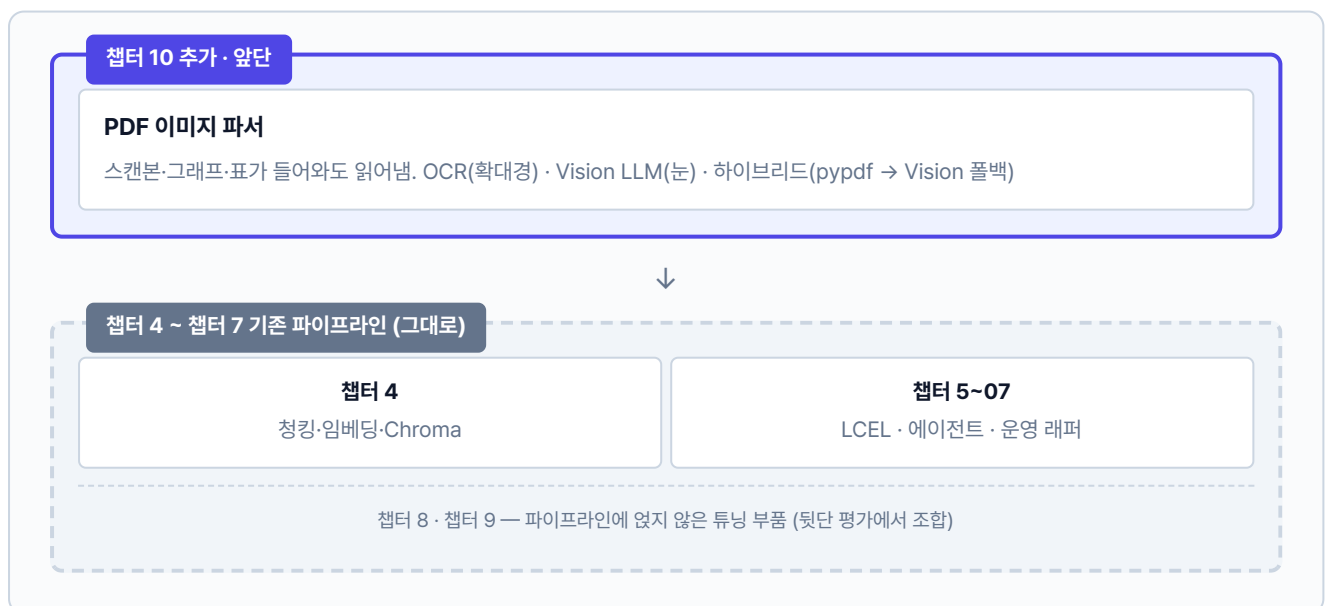
오픈이가 docs 폴더를 훑었습니다. 조직도는 박스와 화살표가 얹힌 이미지, 매출 차트는 막대그래프, 서약서는 스캔본. 사내 문서에는 글자만 있는 게 아닙니다. 이 중 어느 것도 챗터 4의 파서로는 제대로 못 읽습니다.

팀장 : "사서한테 눈을 달아야죠."

한마디가 머리에 걸렸습니다.

기존 파이프라인은 그대로, 앞뒤에 두 층만 엮습니다

챗터 4부터 7까지 쌓아 올린 파이프라인은 이번 챗터에서 건드리지 않습니다. 파싱·청킹·임베딩·검색·에이전트·캐시·모니터링까지 모든 층이 원래 자리에 그대로 있습니다. 챗터 8·9에서 실험한 튜닝(단락 청킹·리랭킹·약어 확장·부모 문서 검색 등)은 아직 이 파이프라인에 얹지 않은 부품 상태로 따로 놓여 있습니다. 이번 장의 할 일은 기존 파이프라인의 앞단과 뒷단에 한 층씩 새로 엮고(PDF 이미지 파서 · RAG 평가 프레임워크), 뒷단에서 그 평가 도구로 챗터 8·9의 부품들을 조합해 어떤 조합이 정말 수치를 끌어올리는지 확인하는 작업입니다. 이 과정을 마치면 커넥트HR 파이프라인은 챗터 7의 기본 형태에서 스캔본까지 읽고 품질을 수치로 검증하는 **새 버전**으로 올라가 다음 챗터의 조립대 위로 넘어갑니다.



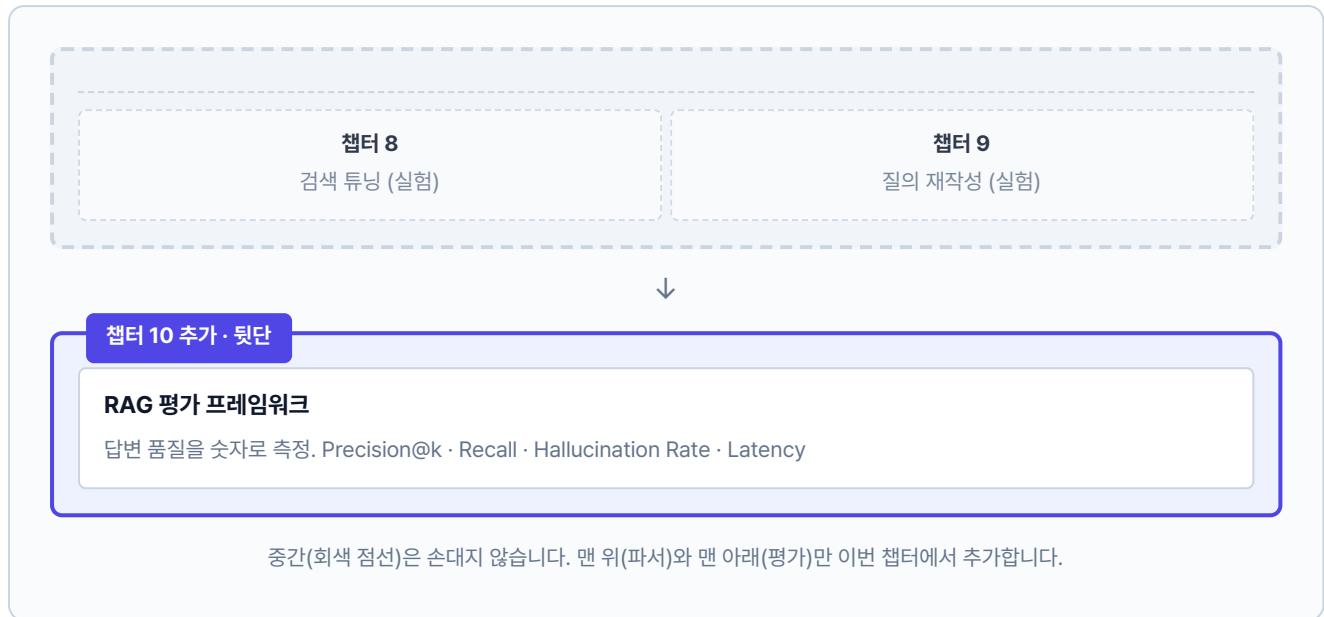


그림 10-2. 챕터 4~07 파이프라인은 그대로 두고 앞뒤에 두 층을 더합니다. 챕터 8·09 튜닝은 뒷단 평가에서 부품으로 조립합니다

10.2 확대경 달기 - OCR

팀장의 말에서 첫 번째 도구가 떠올랐습니다. **확대경**, 즉 **광학 문자 인식(OCR)** 입니다. 이미지 위의 글자를 한 자 한 자 인식해 텍스트로 바꿔 줍니다. 스캔본에서 글자를 꺼낼 때 씁니다.

확대경을 직접 만들어 보겠습니다. `tuning/step1_document_parser/ocr.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 1 tuning/step1_document_parser/ocr.py. EasyOCR로 글자 추출

```
import io
from pathlib import Path
import fitz # PyMuPDF
import numpy as np
from PIL import Image

def parse_pdf_ocr(pdf_path: str | Path, dpi: int = 150) → dict:
    # TODO: EasyOCR로 PDF 페이지를 읽어 텍스트를 추출합니다.
    import easyocr
    # 1. EasyOCR 리더 생성 (한국어+영어)
    reader = easyocr.Reader(["ko", "en"], gpu=False)
    # 2. PyMuPDF로 PDF를 열고 페이지별 이미지 변환
```

```

doc = fitz.open(str(pdf_path))
page_texts = []
for page_num in range(len(doc)):
    page = doc[page_num]
    pix = page.get_pixmap(dpi=dpi)
    img = Image.open(io.BytesIO(pix.tobytes("png")))
    # 3. 이미지를 numpy 배열로 변환 후 OCR 실행
    img_array = np.array(img)
    ocr_results = reader.readtext(img_array, detail=0)
    page_texts.append("\n".join(ocr_results))
doc.close()
return {"text": "\n\n".join(page_texts)}

```

PyMuPDF가 PDF 페이지를 PNG 이미지로 변환해 주면, EasyOCR이 그 이미지에서 글자를 읽어 텍스트 문자열로 돌려줍니다.

두 도구의 역할은 정반대입니다. 하나는 **그려 내는 쪽**이고 다른 하나는 **읽어 내는 쪽**입니다. PDF에 텍스트 레이어가 있으면 pypdf로 바로 꺼내면 그만이지만, 스캔본은 글자 자체가 이미지로 박혀 있어 그 길이 막혀 있습니다. 그래서 한 발 돌아가는 셈입니다. PyMuPDF로 페이지를 고해상도 이미지로 그려 낸 뒤, EasyOCR에게 그 이미지에서 글자를 짚어 보라고 맡기는 두 단계 파이프라인을 탑니다.

PyMuPDF (`import fitz`). PDF 렌더링 라이브러리. 페이지를 원하는 해상도의 이미지로 그려내는 것이 주 역할입니다. 텍스트 레이어가 없는 스캔본도 겉모습(글자 모양·도장의 위치·표의 선)은 언제든지 이미지로 뽑을 수 있습니다.

OCR (Optical Character Recognition, 광학 문자 인식). 이미지 속 글자 모양을 딥러닝 모델로 알아보고 텍스트로 옮기는 기술. **EasyOCR**은 한글을 포함해 80여 개 언어를 기본 지원하는 오픈소스 구현체입니다.

DPI (Dots Per Inch). 1인치에 점을 몇 개 찍어 이미지를 만드는지 나타내는 해상도 단위. 숫자가 클수록 글자가 또렷해지는 대신 이미지가 커지고 처리 시간이 길어집니다.

첫 단계에서 해상도를 정하는 값이 `dpi=150`입니다. 72는 화면에 띄우기엔 거칠고, 300은 인쇄용으로 선명하지만 무겁습니다. 그 사이 150은 OCR이 한글·영문 획을 구분할 만큼은 보이면서 페이지당 이미지 크기를 과하게 키우지 않는 실무 기본값으로 굳어졌습니다.

대상 파일은 `data/docs/hr/HR_정보보안서약서.pdf` (스캔본 1p)입니다. 이 장의 파싱 실험은 전부 이 한 장으로 진행합니다.

터미널 실험 1-1 실행. OCR로 스캔 PDF 읽기

```
python -m tuning.step1_document_parser --step 1-1
```

대상 : HR_정보보안서약서.pdf

전략

OCR (EasyOCR)

추출 글자 수

755자

소요 시간

71.85초

미리보기

가인 승인 Choi Jang 정보보안 서사서 (스스년) 단서면요 2026-HK SEC 002 공기능하 대외비 (Conhidentil)...

글자는 대부분 읽지만 표 구조는 일렬로 늘어섭니다.

그림 10-3. 실험 1-1. OCR 파싱 (EasyOCR)

1분 남짓 걸려 글자 대부분을 읽었지만, 미리보기를 보면 '정보보안 서사서'처럼 한글 오인식이 섞여 있습니다. 표는 더 심각했습니다.

원본 (사람 눈에 보이는 것)

이름	직급	부서
김철수	대리	인사팀
박영희	과장	개발팀

OCR 결과 (확대경이 읽은 것)



이름 직급 부서 김철수 대리 인사팀 박영희
과장 개발팀

셀 구분 없이 일렬로 늘어섭니다

그림 10-4. OCR의 한계. 글자는 읽지만 표 구조가 사라집니다

오픈이는 화면을 잠시 내려다봤습니다. 형광등 빛을 받은 커서가 "정보보안 서사서"라는 일곱 글자 옆에서 조용히 깜빡였습니다. 분명 규정집 첫 페이지에 토박토박 박혀 있던 제목. 기계의 눈을 거치자 '약'이 '사'로, '연'이 '년'으로 얼굴이 바뀌어 돌아와 있었습니다. 표도 마찬가지로. 행과 열이 만들던 격자는 사라지고, 이름과 직급과 부서가 한 줄로 쏟아져 내렸습니다.

이대로 벡터 DB에 들어가면, 질문을 해도 엉뚱한 답이 돌아오겠지.

키보드 위에 얹혀 있던 손을 거두고 서랍에서 수첩을 꺼냈습니다. 터미널은 그대로 열어 둔 채, 방금 눈으로 본 두 가지를 적어 둘 참이었습니다. 한 시간만 지나도 "OCR이 좀 아쉬웠지"라는 인상만 남고, 정확히 어디서 어떻게 깨졌는지는 뭉툭해집니다. **어디서 어떻게 깨졌는지를** 손으로 짚어 두지 않으면, 나중에 다른 도구를 고를 때 같은 함정으로 걸어 들어가기 쉽습니다.

- 실험 메모 -

1. 한글 오인식

- '정보보안 서약서' → '정보보안 서사서'
- '대외비 (Confidential)' → '공기능하 대외비 (Conhidcntil)'
- 도장·고유명사 근처 글자가 뭉개짐

2. 표 구조 소실

- 행·열이 사라지고 한 줄로 늘어섬
- "김철수 대리 인사팀 박영희 과장 개발팀"
- 누가 어느 부서인지 추론 불가

글자는 읽는데, 읽기만 하는구나.

수첩을 덮고 의자 깊숙이 기댔습니다. 화면 속 "스보보안 서의서"와 일렬로 늘어선 이름들이 천천히 위로 올라가다 사라졌습니다. 확대경은 글자를 크게 키워 주긴 했지만, 제 손에 쥐인 것이 서약서인지 출근부인지는 알지 못했습니다. 한 획씩 더듬어 읽는 점자. 딱 그 수준이었습니다.

다른 방법 없을까?

주전자에서 보리차를 따르며 며칠 전 팀장이 지나가듯 던진 말을 곱씹었습니다. "눈을 달아야죠." 그때는 무슨 뜻인지 흘러들었는데, 확대경의 한계를 눈으로 보고 나니 문장이 다르게 들렸습니다. 사람이 문서를 볼 때는 글자 하나하나를 읽기 전에 이미 제목을 알아보고, 표인지 도장인지 구분하고, 네모 칸이 누구를 가리키는지 짐작합니다. 글자를 해독하기 전에 **이해**가 먼저 옵니다.

지금 이 도구에 필요한 건 더 정밀한 확대경이 아니었습니다. 문서를 그림째로 이해해 주는 무언가, 표의 행과 열을 의미로 묶고 "스보보안 서의서"를 "정보보안 서약서"로 복구해 주는 **더 높은 층위의 눈**이 필요했습니다.

10.3 눈 달기 - Vision LLM

두 번째 도구는 눈입니다. 글자를 한 획씩 따라 읽는 눈이 아니라, 이미지 전체를 한 장면으로 이해하는 쪽입니다. **Vision LLM**은 사진·도표·스캔본을 보고 그 안에서 일어나고 있는 일을 말로 설명합니다. "이 차트에서 2024년 매출이 얼마야?"라고 물으면 막대그래프의 높이를 눈으로 재고 축의 숫자를 읽어 답을 돌려 줍니다. 조직도 앞에서는 화살표 방향을 따라가며 "마케팅팀장은 대표에게 보고하고, 그 아래 3명이 있습니다" 식으로 관계를 설명합니다. 표는 셀의 위치를 기억해 행과 열을 다시 짚니다.

OCR이 확대경이었다면, Vision LLM은 눈과 두뇌에 가깝습니다. 확대경은 보이는 것을 크게 보여 줄 뿐이지만, 눈은 본 것을 해석합니다. 문장의 의미를 알고, 문맥으로 오타를 복원하고, 표가 표라는 사실을 이해합니다. 확대경이 놓친 자리에 눈을 대면, 앞 실험에서 뭉개졌던 제목과 무너졌던 격자가 다시 제자리를 찾습니다.

이제 에이전트에 눈을 달아 줄 차례입니다. `tuning/step1_document_parser/vision.py`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 1 `tuning/step1_document_parser/vision.py`. Vision LLM으로 이미지 이해

```
from ._parser_utils import call_vision_llm

def parse_pdf_vllm(pdf_path: str | Path, dpi: int = 150) → dict:
    # TODO: Vision LLM으로 PDF 페이지 이미지를 읽어 텍스트를 추출합니다.
    # 1. PyMuPDF로 PDF를 열고 페이지별 이미지 변환
    doc = fitz.open(str(pdf_path))
    page_texts = []
    for page_num in range(len(doc)):
        page = doc[page_num]
        pix = page.get_pixmap(dpi=dpi)
        # 2. 페이지 이미지를 임시 파일로 저장
        img_path = f"_vllm_page_{page_num + 1}.png"
        pix.save(img_path)
        # 3. Vision LLM에 이미지를 보내 텍스트 추출
        caption = call_vision_llm(img_path)
        page_texts.append(caption)
        Path(img_path).unlink(missing_ok=True)
    doc.close()
    return {"text": "\n\n".join(page_texts)}
```

`call_vision_llm`은 `_parser_utils.py`에 있는 보조 함수로, Ollama의 Vision 모델에 이미지를 보내고 응답을 받습니다. 페이지를 임시 PNG로 저장한 뒤 LLM에 넘기고, 처리가 끝나면 임시 파일을 삭제합니다.

실험 1-1과 같은 `data/docs/hr/HR_정보보안서약서.pdf` (스캔본 1p)를 대상으로 돌려 OCR과 같은 조건에서 결과를 비교합니다.

터미널 실험 1-2 실행. Vision LLM으로 스캔 PDF 읽기

```
python -m tuning.step1_document_parser --step 1-2
```

대상: HR_정보보안서약서.pdf(스캔본 1p) · 모델: qwen2.5vl:7b · DPI: 100

전략

Vision LLM

추출 글자 수

788자

소요 시간

21.86초

미리보기

```
```markdown # 정보보안 서약서 (스캔본) **문서번호:** 2026-HR-SEC-002 **보존연한:** 영구 **
공개등급:** 대외비 (Confidential) ## 제4조 (생성형 AI 활용 지침)...
```

제목·문서번호·조항 번호까지 마크다운 구조로 알려 냅니다.

그림 10-5. 실험 1-2. Vision LLM 파싱

글자 수는 OCR과 비슷한데도 제목과 조항 번호까지 정확히 돌려주고, 마크다운 표 구조가 살아 있어 훨씬 다루기 좋은 결과입니다. 대신 페이지 하나에 22초 남짓 걸렸습니다. 로컬 Vision LLM은 정확한 값을 주는 만큼 연산을 쓰고 있는 셈입니다. (qwen2.5vl:7b, CPU, DPI 100 기준. 더 큰 모델이나 GPU를 쓰면 품질은 올라가고 시간은 줄어듭니다)

오픈이는 모니터 앞으로 몸을 당겼습니다. 파이프 기호와 하이픈으로 그려진 마크다운 표. 줄을 맞춰 서 있었습니다. 조금 전까지 "김철수 대리 인사팀 박영희 과장 개발팀"이라고 한 줄로 쏟아지던 이름들이, 이번에는 이름끼리 직급끼리 각자 제자리로 돌아와 앉았습니다. 제목도 마찬가지. "정보보안 서약서" 여섯 글자가, 방금 전 "정보보안 서사서"로 얼굴이 바뀌어 있던 바로 그 자리에 또렷하게 박혀 있었습니다.

이건 확실히 다르네.

확대경은 점을 하나씩 따라가며 읽었는데, 이 친구는 페이지 전체를 한 번에 훑은 모양이었습니다. 표가 표라는 걸 알고, 제목이 제목이라는 걸 알고, 빈칸 다음에 오는 글자가 문맥상 무엇이어서 하는지까지 짐작해서 채워 넣었습니다.

그런데 감탄 끝에 붙은 조건이 마음에 걸렸습니다. 이 눈이 한 페이지를 읽어 낼 때마다 비싼 값을 치릅니다.

### Vision LLM을 상시로 켜 두기 전에 확인할 비용

- **토큰 비용.** Vision LLM은 한 페이지를 통째로 이미지로 본 뒤 그 안에서 읽어 낸 내용을 토큰으로 쏟아냅니다. 도장·도표가 섞인 스캔본이라면 입력 이미지 토큰만 수천 단위로 불어나고, 답으로 되돌아오는 마크다운 텍스트도 덩달아 쌓입니다. API로 부르면 페이지 수만큼 요금이, 로컬이면 메모리·연산이 그만큼 빠져나갑니다.
- **하드웨어 문턱.** `qwen2.5vl` · `minicpm-v` 같은 7B급 모델은 양자화본 기준 6~8GB 여유 RAM을 요구합니다. CPU만으로도 돌아가지만 페이지당 수십 초 단위로 밀리고, GPU가 있어도 VRAM 8GB 이상이 있어야 안정적입니다. 속도는 단순히 "빠르다/느리다"가 아니라 어떤 머신에서 돌리느냐에 따라 몇 배씩 갈리는 값입니다. (모델 선택 기준은 장 앞쪽 준비하기 tip 참조)
- **중복 비용.** 사내 문서 대부분은 이미 텍스트 레이어가 멀쩡한 PDF입니다. pypdf 한 줄이면 끝날 문서에까지 Vision을 들이밀면 토큰도 메모리도 낭비입니다.

좋은 도구긴 한데, 아무 데나 쓸 건 아니구나.

같은 서약서를 두 도구로 읽은 결과를 나란히 놓으면 차이가 선명합니다.



**OCR**

확대경 (글자는 읽지만 구조는 모릅니다)

표를 읽은 결과

이름 직급 부서 김철수 대리 인사팀 박영희 과장 개발팀

셀 구분 없이 일렬로 늘어섭니다

빠름

저렴

구조 인식 불가



**Vision LLM**

눈 (이미지를 이해합니다)

표를 읽은 결과

| 이름 | 직급 | 부서 |

| 김철수 | 대리 | 인사팀 |

| 박영희 | 과장 | 개발팀 |

표 구조와 관계까지 설명합니다

읽습니다

느림

비쌈

구조+관계 인식

이해는 깊지만 그만큼 토큰과 메모리를 요구합니다. 이미 글자로 잘 꺼낼 수 있는 문서에까지 같은 비용을 치를 이유는 없습니다.

**동료** : "규정집은 텍스트 PDF잖아요. 그것까지 Vision에 돌리면 토큰도 시간도 아깝지 않아요?"

## 10.4 하이브리드 파서 - pypdf가 충분히 읽어 내면 그대로, 아니면 Vision

맞는 말이었습니다. 오픈이가 마우스 휠을 천천히 내리며 docs 폴더를 다시 훑었습니다. 파일 아이콘이 두 갈래로 나뉘어 있었습니다. HR 규정집, 출근 규정, 연차 매뉴얼. 내부에서 워드로 작성해 그대로 PDF로 내보낸 문서들이었습니다. 그 옆에 섞여 있는 정보보안 서약서. 종이에 도장을 찍은 뒤 스캐너를 거친 파일이었습니다.

한 폴더 안에 성질이 다른 두 종류가 있잖아.

두 유형에 같은 도구를 쓰는 것부터 어긋난 접근이었습니다. 규정집까지 Vision에 돌리면 한 줄이면 꺼낼 텍스트를 수십 초에 걸쳐 눈으로 다시 읽는 꼴이 되고, 반대로 pypdf만 고집하면 서약서 앞에서는 손을 놓게 됩니다. 둘 중 하나를 고르는 이분법이 아니라, 페이지마다 상황을 보고 쓰는 도구를 갈라야 했습니다.

오픈이가 의자를 모니터 가까이 당겼습니다. 머릿속에서 그림이 한 장 그려졌습니다. 페이지가 들어오면 일단 pypdf에 맡긴다. 글자가 제대로 돌아오면 그대로. 빈손이거나 한 줄뿐이면 그제야 눈을 호출한다.

전략은 한 줄이었습니다.

**pypdf가 꺼낸 텍스트가 충분하면 그대로 쓰고, 아니면 눈을 부른다.**

페이지마다 `page.get_text()` 로 텍스트를 꺼낸 뒤, 길이가 50자 이상이면 pypdf 결과를 그대로 쓰고, 그보다 적으면 스캔본이나 차트 페이지로 보고 Vision에 넘깁니다. 판단은 페이지 단위로 독립이라, 10페이지짜리 문서에 스캔 페이지가 1장만 섞여 있어도 그 1장만 Vision으로 가고 나머지는 pypdf가 즉시 끝냅니다.

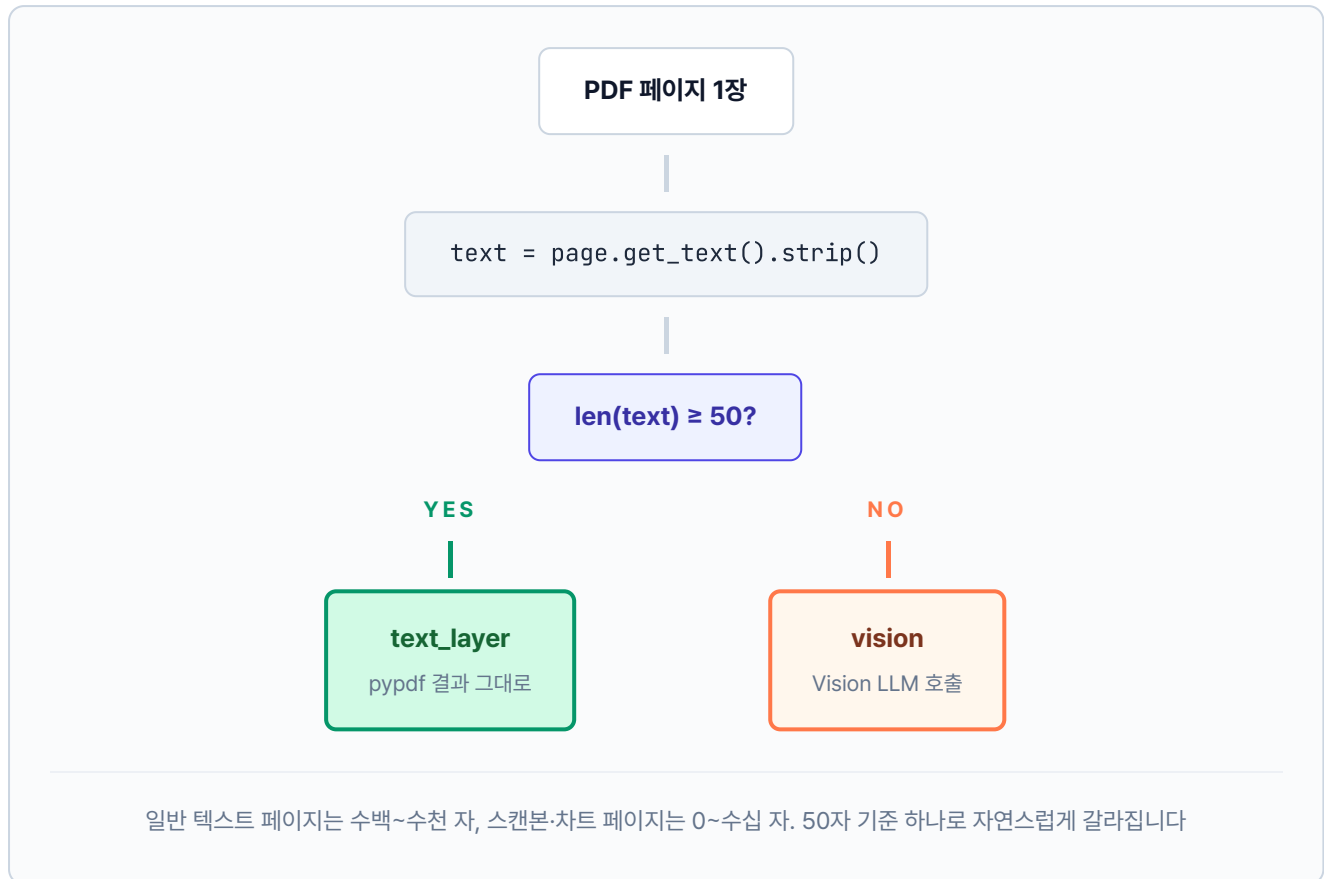


그림 10-7. 하이브리드 파서의 판단 흐름. 페이지마다 텍스트 길이로 분기합니다

간단하게 만들어 보겠습니다. `tuning/step2_hybrid_parser/hybrid_parser.py`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

**실습 2** `tuning/step2_hybrid_parser/hybrid_parser.py`. pypdf 결과로 파서 선택

```

import fitz # PyMuPDF
from ._hybrid_utils import vision_page

MIN_TEXT_LENGTH = 50 # 이 미만이면 스캔본 or 차트 페이지로 간주하고 Vision

def process_page_hybrid(
 page: fitz.Page,
 dpi: int = 150,
 vision_model: str | None = None,
) → dict:
 # TODO: pypdf 텍스트가 충분하면 text_layer, 부족하면 Vision.
 # 1. pypdf 텍스트 레이어 추출
 text = page.get_text().strip()
 # 2. 텍스트 충분 → 그대로 사용 (밀리초 단위)
 if len(text) ≥ MIN_TEXT_LENGTH:

```

```

 return {"strategy": "text_layer", "text": text, "char_count": len(text)}
3. 텍스트 부족 → 스캔본 or 차트 페이지. Vision으로 전환
vision_text = vision_page(page, dpi=dpi, model=vision_model)
 return {"strategy": "vision", "text": vision_text, "char_count": len(vision_text)} if vision_text else 0}

```

`page.get_text()` 는 PDF 페이지의 텍스트 레이어를 꺼내는 한 줄이고, `MIN_TEXT_LENGTH` 임계값은 "이 정도도 안 나오면 그림이 자리를 차지하고 있다"는 판단 기준입니다. 스캔본은 0자가 돌아오니 곧장 Vision으로 넘어가고, 텍스트 PDF는 수백 자가 돌아와 그 자리에서 끝납니다. 네이티브 벡터 차트 페이지도 축 레이블 몇 글자 정도라 50자 아래로 떨어지고, 결국 Vision 차례가 됩니다. 10.2절에서 한계를 확인한 OCR은 이 조합에 넣지 않았습니다.

### 이 하이브리드의 경계와 한 걸음 더

이번 파서는 **페이지 단위 텍스트 길이 하나로** 판단합니다. 그래서 커버하는 범위와 남는 한계가 명확합니다.

- **잘 잡는 것:** 스캔본(전체가 이미지), 일반 텍스트 PDF, 네이티브 벡터 차트로만 이뤄진 페이지. 이 챕터의 테스트 문서 전부가 여기 해당합니다.
- **놓치는 것:** 한 페이지 안에 본문 텍스트 + 작은 차트가 섞여 있는 경우. 본문이 50자를 훌쩍 넘기니 `text_layer` 로 분기되고, 차트의 막대 높이·선 기울기 같은 시각 정보는 손실됩니다. 축 레이블·범례 텍스트만 엉뚱하게 남습니다.
- **한 걸음 더:** 이 한계를 넘으려면 페이지를 **영역(bounding box) 단위로 쪼개서** 텍스트 영역은 pypdf, 차트 영역은 Vision으로 따로 보내야 합니다. 실무에서는 `unstructured`, `docling` (IBM), `LayoutParser` 같은 레이아웃 분석 도구가 이 역할을 맡습니다. 대신 설치·학습·디버깅이 한 단계 무거워집니다.
- **언제 확장하나:** 사내 보고서·기획서처럼 본문과 차트가 같은 페이지에 뒤섞인 문서 비중이 의미 있게 커지면 그때 레이아웃 분석 도구를 도입하세요. 스캔본·텍스트 PDF가 대부분이라면 지금 하이브리드로 충분합니다.

**터미널** 실험 2 실행. 하이브리드 파서로 페이지별 자동 분기

```
python -m tuning.step2_hybrid_parser
```

대상: HR\_정보보안서약서.pdf(스캔본 1p) · HR\_취업규칙\_v1.0.pdf(텍스트 1p) · Vision: qwen2.5vl:7b  
· DPI 100

페이지	전략	글자 수
서약서 p.1	vision	788
취업규칙 p.1	text_layer	1,426
전략 분포 vision 1/2 · text_layer 1/2		총 소요 시간 91.94초
스캔본은 Vision이, 텍스트 PDF는 pypdf가 맡습니다. 한 파이프라인이 두 경로를 자동 선택합니다.		

그림 10-8. 실험 2. 하이브리드 파싱

오픈이가 실행 버튼을 누르고 터미널을 지켜봤습니다. 노트북 팬이 제법 오래 숨을 몰아쉬더니 첫 줄이 올라왔습니다. `서약서 p.1 ... vision`. 한 페이지를 읽는 데 1분 30초 남짓. 그러다가 다음 줄에서 속도가 똑 바뀌었습니다. `취업규칙 p.1 ... text_layer`, 1,426자, 0.17초. 깜빡이 한 번에 끝이었습니다.

파이프라인이 페이지를 한 장씩 들춰 본 다음 스캔본에는 눈을 대고, 텍스트 PDF는 pypdf로 곧장 꺾어 낸 모양새였습니다. 총 91.94초 가운데 91.77초가 Vision 한 번에 쓰였고, 텍스트 한 페이지는 사실상 공짜로 빠졌습니다. 필요할 때만 비싼 도구를 부르는 구조가 숫자로 그대로 드러났습니다. (Vision 시간은 모델을 처음 메모리에 올리는 콜드 스타트를 포함한 값입니다. 한 번 예열되면 호출당 20초 남짓으로 줄어듭니다)

되긴 됐다.

오픈이가 의자 등받이에 몸을 기댔습니다. 어깨에 들어가 있던 힘이 천천히 빠졌습니다. 고개를 돌려 옆자리 팀장을 불렀습니다.

**오픈이** : "팀장님, 서약서 건 돌아갑니다. 스캔본은 Vision으로, 규정집은 pypdf로 페이지마다 알아서 갈라져요."

팀장이 의자를 밀며 다가와 터미널 로그를 훑어봤습니다.

**팀장** : "오, 둘 다 한 파이프라인에서 처리되네요."

**오픈이** : "네. 글자가 충분히 나오면 그대로 쓰고, 부족하면 그때 Vision을 붙이는 식이에요."

팀장이 고개를 몇 번 끄덕이더니 로그를 처음부터 끝까지 한 번 더 눈으로 따라갔습니다. 그러고는 의자에서 일어서며 한 마디 던졌습니다.

**팀장** : "이게 얼마나 좋아진 건지는 어떻게 알아요?"



## 10.5 RAG 평가: 느낌이 아니라 숫자

바로 대답이 나오지 않았습니다. 오픈이가 잠시 시선을 내리더니 수첩을 펼쳤습니다. 챗터 8부터 여기까지 손본 튜닝이 한 페이지를 가득 메우고 있었습니다.

- 지금까지 쌓인 튜닝 -

1. 청킹
  - 의미 기반 분리 (챗터 8)
2. 리랭커 (챗터 8)
  - CrossEncoder 재정렬
3. 하이브리드 검색 (챗터 8)
  - BM25 + 벡터 유사도
4. HyDE (챗터 9)
  - 가상 답변을 먼저 만들어 검색
5. 부모 문서 검색 (챗터 9)
  - 자식 청크로 찾고 부모 페이지 반환
6. 약어 확장 (챗터 9)
  - WFH → 재택근무
7. 하이브리드 파서 (챗터 10)
  - pypdf 부족하면 Vision 전환

생각해 보니 이 파서 하나만이 아니네. 지금까지 엮은 것들도 다 "좋아졌다"고만 말했지, 얼마나 기여했는지는 모른다.

체감만 있고 근거가 없었습니다. 튜닝을 더 엮을지, 이쯤에서 멈출지, 어느 조합으로 갈지. 전부 숫자가 뒤를 받쳐 줘야 답할 수 있는 질문이었습니다. 팀장의 한마디는 이번 파서 하나를 묻고 있는 것이 아니었습니다. 지금까지 쌓인 튜닝 전부에 대한 답을 요구하고 있었습니다.

평가 프레임워크를 꺼낼 차례였습니다.

평가 프레임워크는 RAG 시스템의 품질을 숫자로 재는 채점기입니다. 같은 질문 셋을 여러 튜닝 조합에 던지고, 미리 정해 둔 지표로 결과를 찍어 비교하는 구조입니다. 선생님이 학생들의 시험지를 같은 채점표로 내려 점수를 매기듯, 여러 조합을 같은 잣대 위에 놓아야 어느 튜닝이 정말 값을 하는지 가려낼 수 있습니다.

먼저 어떤 지표로 잴지 정해야 합니다. 이 책에서는 서로 다른 각도를 잡아 줄 네 가지로 추렸습니다.

- **Precision@k** — "검색이 정답을 위에 잘 올렸나?" (상위 k칸의 정확성)
- **Recall@k** — "정답을 빠짐없이 잡았나?" (정답 커버리지)
- **Hallucination Rate** — "답변이 근거 문서 안에서 나왔나?" (지어내기 억제)
- **Latency** — "사용자가 기다릴 만한 속도인가?" (응답 시간)

Precision과 Recall은 검색이 어떻게 돌아가는지를 들여다보고, Hallucination Rate는 만들어진 답변을 뜯어 보고, Latency는 사용자가 실제로 겪는 대기 시간을 잹니다. 한 지표만 붙들면 다른 쪽이 무너져도 모르고 지나칩니다. Recall만 끌어올리다 보면 관련 없는 문서까지 딸려 와 환각이 늘어나고, 속도만 좇으면 품질이 뒷줄로 밀립니다. 네 지표를 한 세트로 놓고 봐야 전체 모양이 보입니다.

지표	의미	계산
<b>Precision@k</b>	상위 k개 중 정답 비율	$(\text{정답 체크 수}) / k$
<b>Recall</b>	전체 정답 중 발견한 비율	$(\text{발견한 정답 수}) / (\text{전체 정답 수})$
<b>Hallucination Rate</b>	답변 중 문서 근거 없는 문장 비율	답변과 근거 문서의 단어 겹침 비율 (더 정확히 잴 땐 별도 LLM에게 채점을 맡깁니다)
<b>Latency</b>	질문당 평균 응답 시간	ms 단위

#### 이 네 지표는 업계 표준과 같은 축을 잹니다

Precision@k-Recall@k는 **문서 검색** 분야에서 오래 쓰인 교과서 지표이고, Hallucination Rate는 RAG가 등장한 뒤 자리 잡은 항목, Latency는 운영 지표의 기본입니다. RAGAS·LangSmith 같은 오픈소스 RAG 평가 도구도 이름만 살짝 바꿔 같은 네 축(혹은 그 변형)을 잹니다. 뒤 10.5.3에서 "이 책의 간단한 구현을 실무에선 어떻게 더 정밀하게 바꾸는지"를 다시 짚습니다.

오픈이가 새 파일을 하나 만들었습니다. `data/test_questions.json`. 비어 있는 배열. 대괄호 사이에서 커서가 조용히 깜빡였습니다.

## 뭘 기준으로 물어보지?

동료들이 이제까지 던진 질문들을 떠올려봤습니다. "연차 신청은 어떻게 해요?", "병가 며칠까지 돼요?", "USB 써도 돼요?", "이번 분기 매출 얼마예요?" 기억나는 것들을 한 줄씩 적어 내려갔습니다. 매 질문 옆에는 이 답이 어느 문서에서 나와야 하는지, 이상적인 답변은 어떤 내용인지를 함께 적어 뒀습니다.

질문 31개가 쌓였습니다. 각 건은 `query` (테스트 질문), `relevant_sources` (그 답이 담겨 있어야 할 원본 문서명 리스트), `expected_answer` (이상적 답변) 세 축. 앞 두 필드는 Precision@k·Recall 계산에, 마지막 필드는 Hallucination Rate 판정에 쓰입니다.

이게 우리 시스템의 채점지네.

`tuning/step3_eval_framework/metrics.py` 를 열면 Precision@k와 환각률 함수 두 개가 TODO로 비어 있습니다. 하나씩 채워 나갑니다.

### 10.5.1 Precision@k — 상위 k개에 정답 문서가 얼마나 올라왔나

첫 번째 지표는 **Precision@k**. 시스템이 돌려준 검색 결과 상위 k개 중에 정답 문서가 몇 개 들어 있는지를 비율로 재는 지표입니다. 채점지로 치면 "답안지 첫 k칸 중 정답이 몇 칸인가"를 세는 항목입니다. 오픈이가 JSON에 적어 둔 `relevant_sources` 가 정답지가 되고, 시스템이 돌려준 검색 결과 문서명이 답안지가 됩니다.

`calculate_precision_at_k` 함수의 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

**실습 3** `tuning/step3_eval_framework/metrics.py`. Precision@k 계산

```
def calculate_precision_at_k(
 retrieved_sources: list[str],
 relevant_sources: list[str],
 k: int,
) → float:
 # TODO: 상위 k개 결과 중 정답 문서명을 포함한 비율을 반환합니다.
 # 1. 상위 k개만 추린다
 top_k = retrieved_sources[:k]
 # 2. 정답 문서명은 set으로 묶어 조회 속도를 O(1)로 만든다
 relevant_set = set(relevant_sources)
 # 3. 각 top-k 결과 source가 정답 문서명 중 하나라도 포함하는지 센다
 hits = sum(1 for src in top_k if any(rel in src for rel in relevant_set))
 # 4. k로 나눠 비율을 돌려준다 (k=0 방어)
 return hits / k if k > 0 else 0.0
```

매개변수·동작을 풀면 다음과 같습니다.

매개변수	역할
<code>retrieved_sources</code>	시스템이 검색해 가져온 청크의 source 문서명 리스트. 순서가 중요 (위에 올린 것부터 k번째까지만 본다)
<code>relevant_sources</code>	JSON에 적어 둔 정답 문서명 리스트
<code>k</code>	상위 몇 개까지 채점할지 정하는 숫자 (보통 3 또는 5)

문자열 부분 일치(`rel in src`)로 판단한 데는 이유가 있습니다. 검색 결과 source에는 `_chunk_5` 같은 청크 번호 접미사가 붙을 때가 많아서, 정답 문서명이 포함되기만 하면 맞다고 처리해야 합니다. 결과는 0에서 1 사이 값이고, 1에 가까울수록 상위 k 안에 정답 문서가 많이 들어와 있다는 뜻입니다.

예시를 따라가 보겠습니다. `retrieved_sources = ['HR_취업규칙_v1.0_chunk2', 'SEC_보안규정_v1.0_chunk1', 'HR_취업규칙_v1.0_chunk7']`, `relevant_sources = ['HR_취업규칙_v1.0']`, `k=3` 일 때, 상위 3개 중 2개가 `HR_취업규칙_v1.0` 을 포함하니 `Precision@3 = 2/3 ≈ 0.67` 이 됩니다.

그럼 0.67이라는 숫자는 무엇을 의미하는 걸까요. 상위 3칸 중 약 67%가 정답 문서였다는 얘기입니다. 1.0이면 상위 k칸이 전부 정답 문서로 채워진 이상적인 상태, 0.0이면 상위 k칸 어디에서도 정답을 찾지 못한 상태입니다. 실무에서는 0.7 이상이 나오면 검색이 꽤 잘 맞추고 있다고 보고, 0.5 아래로 떨어지면 튜닝이 필요하다는 신호로 읽습니다. 이 숫자 하나로 "검색이 정답을 위에 얼마나 잘 올렸나"를 비교할 바탕이 생긴 셈입니다.

### 10.5.2 Recall@k — 정답 문서를 빠짐없이 찾았나

**Recall@k**는 Precision@k의 반대편에서 같은 검색 결과를 바라봅니다. Precision이 "답안지에 정답이 몇 %냐"를 물었다면, Recall은 "정답지의 문서를 몇 개나 찾았느냐"를 묻습니다. 정답 문서를 놓치는 일이 잦을수록 점수가 떨어지는 지표입니다.

같은 파일의 `calculate_recall_at_k` 함수 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

**실습 3** tuning/step3\_eval\_framework/metrics.py. Recall@k 계산

```
def calculate_recall_at_k(
 retrieved_sources: list[str],
```

```

relevant_sources: list[str],
k: int,
) → float:
 # TODO: 정답 문서 중 상위 k개 안에 들어온 비율을 반환합니다.
 # 1. 상위 k개만 추린다
 top_k = retrieved_sources[:k]
 # 2. 정답 문서명을 set으로 묶는다
 relevant_set = set(relevant_sources)
 # 3. 각 정답 문서명이 top-k 결과 중 하나라도 포함되는지 센다
 hits = sum(1 for rel in relevant_set if any(rel in src for src in top_k))
 # 4. 정답 문서 개수로 나눠 비율을 돌려준다
 return hits / len(relevant_set) if relevant_set else 0.0

```

Precision@k와 코드가 거의 똑같아 보이지만, **나눗셈의 분모**가 다릅니다.

지표	보는 방향	분모
Precision@k	답안지에 정답이 얼마나?	k (답안 칸 수)
Recall@k	정답 중 얼마나 찾았나?	정답 문서 개수

예시로 감을 잡아 보겠습니다. 정답 문서가 ['HR\_취업규칙\_v1.0'] 하나인데 상위 3개 중 2개가 이 문서 청크라면,  $\text{Precision@3} = 2/3 \approx 0.67$  이지만  $\text{Recall@3} = 1/1 = 1.0$  입니다. 답안지에 오답이 하나 섞였어도 정답은 빠짐없이 잡아낸 셈입니다. Precision은 답안의 정확성을, Recall은 빠짐없는 정도를 본다고 기억해 두면 됩니다.

Precision과 Recall 두 지표는 검색 쪽 성적표 역할을 합니다. 상위 k개에 정답이 얼마나 들어 있는지 (정확성), 정답이 얼마나 빠짐없이 잡혔는지(커버리지)를 각각 보여 줍니다. 다만 검색을 잘했다고 답까지 맞다는 보장은 없습니다. 올바른 문서를 펴 놓고도 내용을 엉뚱하게 옮기면 답은 틀어집니다. 이번엔 답변 자체를 채점할 차례입니다.

### 10.5.3 환각률 — 답변이 근거 문서에서 벗어났다

두 번째 지표는 **환각률**. 시스템이 문서에 없는 내용을 지어내진 않았는지 재는 지표입니다. 학생이 시험에서 문제지에 없는 답을 써내는 상황을 떠올리면 됩니다.

가장 정확한 방법은 또 다른 LLM을 심사위원으로 세워 문장마다 판정하게 하는 것이지만, 로컬 환경에선 너무 무겁습니다. 그래서 이 책은 **가벼운 방식**을 씁니다. 답변에서 핵심 단어를 뽑아 근거 문서에 실제로 등장하는지 센 다음, 너무 적게 나오면 "문서에서 벗어난 답"으로 판정하는 방식입니다.

같은 파일의 `estimate_hallucination_rate` 함수 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

**실습 3** tuning/step3\_eval\_framework/metrics.py. 환각률 추정

```
def estimate_hallucination_rate(
 answers: list[str],
 contexts: list[list[str]],
) → float:
 # TODO: 답변이 컨텍스트에 근거하지 않는 비율을 추정합니다.
 hallucination_count = 0
 # 1. 각 (답변, 컨텍스트) 쌍을 순회한다
 for answer, context_docs in zip(answers, contexts):
 # 2. 컨텍스트 문서들을 하나의 소문자 문자열로 합친다
 context_combined = " ".join(context_docs).lower()
 # 3. 답변에서 4자 이상 단어만 뽑아 핵심 단어 후보로 삼는다
 key_words = [w for w in answer.lower().split() if len(w) > 3]
 if key_words:
 context_words = set(context_combined.split())
 # 4. 핵심 단어 중 컨텍스트에 등장하는 비율이 30% 미만이면 환각으로 카운트
 overlap = len([w for w in key_words if w in context_words]) / len(key_words)
 if overlap < 0.3:
 hallucination_count += 1
 # 5. 환각 답변 수를 전체 답변 수로 나눠 비율 반환
 return hallucination_count / len(answers) if answers else 0.0
```

매개변수	역할
<code>answers</code>	시스템이 낸 답변 문자열 리스트
<code>contexts</code>	각 답변에 쓰인 근거 문서 조각들의 리스트 (답변당 여러 조각이 묶임)

4자 이상 단어만 뽑는 건 한국어 조사나 짧은 단어가 비율을 흐리지 않게 하려는 장치입니다. 30% 기준은 "답변 핵심 단어 10개 중 3개도 문서에 못 찾으면 수상하다"고 보는 실무 감각에서 나온 숫자입니다. 결과는 0에서 1 사이의 비율이고, 0에 가까우면 시스템이 문서에 충실한 쪽, 1에 가까우면 지어낸 답이 많은 쪽입니다.

이 방식의 한계는 **표현만 바꾼 답**을 잡지 못한다는 점입니다. 문서에 "연간 30일"이라고 써 있는데 답변이 "한 해에 30일"로 바꿔 쓰면, 단어가 달라졌다는 이유로 환각으로 오판할 수 있습니다. 더 높은 정밀도가 필요한 실무에선 별도의 LLM에 판정을 맡기는 방식으로 갈아타는 게 좋습니다.

### 이 책의 환각률 측정을 실무에선 이렇게 업그레이드합니다

- **간단한 단어 겹침 대신 LLM 채점:** 이 책의 환각률은 "답변 단어가 근거 문서에 얼마나 겹치나"를 세는 간단한 방식입니다. 실무에선 보통 **별도 LLM에게 문장별로 채점을 맡기는 방식**(업계에선 이 접근을 LLM-as-judge라고 부릅니다)으로 바꿉니다. 단어가 달라져도 뜻이 같으면 옳은 답으로 인정해 줘서 오판이 줄어듭니다.
- **또는 의미 유사도 비교:** 문장을 벡터로 바꾼 뒤 답변 문장과 근거 문장의 벡터가 얼마나 가까운지 잹니다. 챗터 4의 임베딩을 똑같이 씁니다. 단어 겹침보다 "표현만 바꾼 답"에 강합니다.
- **대표 평가 도구:**
  - **RAGAS:** 가장 많이 쓰이는 오픈소스. Faithfulness(답변이 근거에 충실한가) · Answer Relevancy(질문에 잘 답하는가) · Context Precision · Context Recall 네 축을 자동 측정
  - **TruLens:** Groundedness · Context Relevance · Answer Relevance 세 축("RAG Triad"라는 이름으로 통용)
  - **LangSmith:** LangChain의 실행 기록·평가 플랫폼. 서비스에 올라간 뒤 들어오는 요청 로그와 평가를 한자리에서 봅니다
  - **DeepEval:** `pytest` 로 단위 테스트를 쓰듯 RAG 품질을 테스트. 자동 빌드·테스트 환경(CI)에 붙이기 쉽습니다
  - **Arize Phoenix:** 오픈소스 실행 기록 + 평가 도구. 결과를 그래프 대시보드로 보여주는 게 강점
- **출발점 추천:** 지표만 빠르게 보고 싶다면 RAGAS, LangChain으로 시스템을 만들었다면 LangSmith, 자동 테스트 중심이면 DeepEval이 시작점으로 좋습니다.

### 10.5.4 조합별 측정 - 튜닝을 플러그처럼 갈아끼운다

지표는 준비됐지만 아직 절반입니다. 같은 질문 셋을 돌려 보려면 **튜닝 조합을 갈아끼우는 스위치**가 있어야 합니다. 파싱·청킹·쿼리 변환·검색·리랭크 같은 조각을 이름별로 하나씩 꺼내 써서, 조합마다 파이프라인 하나를 조립하는 구조가 필요합니다.

오픈이가 `step3_eval_framework/pipelines.py` 에 부품 함수들을 모았습니다. 한 함수가 한 조각을 담당합니다. 그리고 `strategies.py` 에서 이 부품들을 묶어 A/B/C/D 네 조합으로 정의했습니다. D 조합(챗터 8~10 튜닝이 모두 켜진 최종 형태)을 기준으로, 파이프라인의 네 단계에 어떤 튜닝이 어디에 붙는지 한 장에 담았습니다.



그림 10-9. 튜닝 로드맵. 네 단계를 ㄴ자로 잇고 각 단계에 어떤 튜닝이 붙는지 표시합니다

각 조합(A/B/C/D)은 위 네 단계에서 **얼마나 많은 튜닝 스위치를 켜는지**만 다릅니다. A는 한 개도 켜지 않은 baseline, D는 모두 켜 최종 형태입니다.

참고 tuning/step3\_eval\_framework/strategies.py. 조합 정의

```
from dataclasses import dataclass
from . import pipelines as P

@dataclass(frozen=True)
class Strategy:
 name: str
 description: str
 parse_fn: callable
```



```

retrieve_fn: callable
rerank_fn: callable

STRATEGIES = {
 "A": Strategy("A (baseLine)", "페이지 청킹 + 벡터 검색",
 P.parse_pypdf, P.chunk_page, P.query_noop,
 P.retrieve_cosine, P.rerank_noop),
 "B": Strategy("B", "단락 청킹 + 키워드 리랭킹",
 P.parse_pypdf, P.chunk_semantic, P.query_noop,
 P.retrieve_cosine, P.rerank_keyword),
 "C": Strategy("C", "B + 약어 확장 + Parent Doc",
 P.parse_pypdf, P.chunk_semantic, P.query_expand_abbreviations,
 P.retrieve_parent_doc, P.rerank_keyword),
 "D": Strategy("D", "C + 하이브리드 파서",
 P.parse_hybrid, P.chunk_semantic, P.query_expand_abbreviations,
 P.retrieve_parent_doc, P.rerank_keyword),
}

```

`pipelines.py`에는 각 부품의 실제 로직이 들어 있습니다. 예를 들어 하이브리드 파서를 갈아끼우는 부품은 `step2_hybrid_parser`에서 이미 만든 `process_page_hybrid`를 그대로 재사용합니다.

**참고** tuning/step3\_eval\_framework/pipelines.py. 하이브리드 파서 부품

```

def parse_hybrid(pdf_path) → list[str]:
 """pypdf 텍스트가 부족하면 Vision으로 전환. step2_hybrid_parser 재사용."""
 from ..step2_hybrid_parser.hybrid_parser import process_page_hybrid
 doc = fitz.open(str(pdf_path))
 pages = []
 for page in doc:
 result = process_page_hybrid(page)
 pages.append(result.get("text", ""))
 doc.close()
 return pages

```

`evaluator.run_evaluation(k, strategy_name)`이 `STRATEGIES`에서 조합을 꺼내 부품을 순서대로 호출합니다. A를 돌리면 A 파이프라인이, D를 돌리면 D 파이프라인이 동일 질문 셋에 적용되어 지표가 나옵니다.

**터미널** 조합별 평가 실행

```

조합 하나만
python -m tuning.step3_eval_framework --strategy D --k 3

```

```
네 조합을 한 번에 (그림 10-10 비교표 스타일로 출력)
python -m tuning.step3_eval_framework --strategy all --k 3
```

`--strategy all` 을 쓰면 A부터 D까지 차례로 벡터DB를 다시 짓고 같은 질문 셋을 돌려 네 줄짜리 비교표를 터미널에 찍어 줍니다.

같은 질문 31개로 4가지 튜닝 조합을 비교한 실측 결과 표입니다. 환경: data/docs 문서 6건 (PDF 3 · DOCX 1 · XLSX 2), k=3, Vision: qwen2.5vl:7b, 로컬 임베딩 (ko-sroberta-multitask)

조 합	구 성	Precision@3	Recall@3	환 각 른	Latency
A (baseline)	페이지 청킹 + 벡터 검색	0.204	0.452	0.871	78 ms
B	단락 청킹 + 키워드 리랭킹	0.333	0.323	1.000	78 ms
C	B + 약어 확장 + Parent Doc	0.247	0.548	0.903	85 ms
D	C + 하이브리드 파서	0.236	0.532	0.871	80 ms

환각률은 D가 최저(0.871). Recall은 C·D가 베이스라인 A를 0.08~0.10 끌어올렸고, 스캔본까지 본문으로 편입되는 조합은 D뿐입니다

그림 10-10. RAG 평가 — 조합별 비교

### 실측표를 어떻게 읽을까

- **Precision@3이 낮게 보이는 이유:** 6건짜리 문서 세트는 질문당 정답 후보가 보통 한 건입니다.  $k=3$ 으로 세 칸을 다 채울 때 앞 한 칸만 맞아도 Precision은  $1/3(\approx 0.33)$ 이 위쪽 한계입니다. 표의 B가 그 한계에 거의 맞닿아 있습니다.
- **Recall@3을 최우선 지표로:** 이 시스템은 "정답 문서를 놓치지 않고 올리느냐"가 먼저입니다. C·D가 A(0.452)보다 0.08~0.10 높고, 약어 확장·부모 문서 복원·Vision 편입이 검색 커버리지를 실제로 넓혔다는 뜻입니다.
- **환각률이 왜 0.87~1.00인가:** `estimate_hallucination_rate` 는 `expected_answer` (문서에 없는 가상 정답)와 검색된 청크의 단어 겹침만 봅니다. 예제용 6건 문서 세트에는 이 가상 수치가 그대로 담겨 있지 않아 겹침이 낮게 나옵니다. 수치의 절대값보다 조합 간 상대 비교로 읽으세요.
- **D가 환각률에서 앞서는 이유:** qwen2.5vl이 스캔본에서 "정보보안 서약서" "제4조 (생성형 AI 활용 지침)"처럼 원문 용어를 정확히 복원하기 때문에 검색된 청크에 질문 키워드가 더 자주 등장합니다. 이 차이가 환각률을 C의 0.903에서 D의 0.871로 내렸습니다.

오픈이가 표를 한참 들여다봤습니다. D 행의 환각률 `0.871` 이 눈에 들어왔습니다. 같은 줄의 `0.903` (C), `1.000` (B)보다 한 단계 낮은 숫자. Recall은 C와 0.016 차이로 거의 붙어 있고, Latency도 80 ms 대로 체감 차이 없는 범위였습니다. 대신 D 쪽에는 다른 조합에는 없는 무기가 있었습니다. **스캔본 서약서의 본문이 벡터DB에 들어와 있다는 것.** A·B·C는 서약서 페이지를 아예 인덱싱하지 못했는데, D는 Vision으로 788자를 꺼내 집어넣었습니다.

커서가 D 행 위에서 멈췄습니다.

**오픈이 :** "팀장님, D가 환각률이 제일 낮아요. 서약서 본문까지 올라와 있어서 검색 결과 청크에 질문 단어가 더 자주 등장하는 거예요."

팀장이 잠깐 화면을 내려다보더니 고개를 끄덕였습니다.

**팀장 :** "스캔본이 한 장뿐인데도 숫자에 반영되네요. 종류가 다른 문서를 한 파이프라인으로 다 받는다는 점에서도 D가 맞는 웃이예요."

의자 등받이에 몸이 닿는 순간, 머릿속에서 다른 정리가 났습니다.

튜닝은 지표 하나를 위로 올리는 게 아니라, 쌓아 둔 부품이 서로 물려 돌아가는지 확인하는 작업이구나.

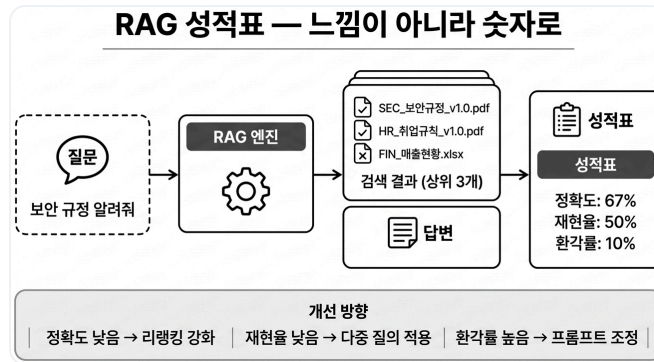


그림 10-11. 평가 프레임워크. 질문과 정답 쌍을 반복 돌려 숫자로 품질을 측정합니다

## 10.6 튜닝 메뉴판 - 무엇을 골라 쓸까

평가 결과에서 고른 D 조합을 실제로 어떻게 조립할지, 챕터 8~10에 등장한 튜닝을 한 줄씩 놓고 재점검할 차례였습니다. 오픈이가 화이트보드에 튜닝 이름을 하나씩 적어 내려갔습니다.

**오픈이** : "전부 다 켜면 좋겠지만, 느려지고 복잡해집니다. 평가에서 숫자로 증명된 것만 남기겠습니다."

**팀장** : "규정 문서 성격 그대로 가면 돼요. 조항이 짧고, 약어가 많고, 스캔본도 일부 섞여 있고요."

튜닝	무엇을 바꾸나	언제 쓰나	LLM 호출	챗터
✓ 청킹 전략	문서 자르는 방법 교체	주제 섞일 때	없음	8
✓ 리랭킹	Cross-Encoder 재정렬	상위 결과 부정확	없음 (reranker 모델)	8
✓ 하이브리드 검색	Vector+BM25 합산	의미+키워드 둘 다	없음	8
HyDE	상상 답변으로 검색	어휘 차이 클 때	있음	9
Multi-Query	질문 여러 갈래	의미가 넓을 때	있음	9
✓ 약어/동의어 확장	WFH→재택근무	약어 많을 때	없음 (사전 치환)	9
✓ Parent Document	자식 청크 검색 → 부모 반환	맥락이 짧게 잘릴 때	없음	9
Compression	핵심 문장만 압축	토큰 절약	있음	9
✓ Vision LLM 파서	스캔PDF/이미지 읽기	스캔본이 섞여 있을 때	있음 (Vision)	10

그림 10-12. 챗터 8~10 튜닝 메뉴판. 체크 표시가 커넥트HR 에이전트에 적용한 일곱 조합입니다

**오픈이** : "Semantic 청킹으로 단락별로 자르고, 리랭커로 순위를 보정하고, 검색은 BM25와 벡터를 섞은 하이브리드로 넓혀 두고, 약어 사전으로 WFH 같은 말을 풀어 주고, Parent Doc으로 짧은 청크에 맥락을 엮습니다. 스캔본은 Vision 파서로 텍스트를 꺼내 같은 벡터DB에 태우고요. 파이프라인 밖에서는 RAG 평가 프레임워크로 주기적으로 질문셋을 돌려 이 일곱 가지가 실제로 효과가 있는지 숫자로 확인합니다."

**팀장** : "HyDE랑 Multi-Query, Compression은 왜 뺐어요?"

**오픈이** : "셋 다 질문이 들어올 때마다 LLM을 한 번씩 더 부릅니다. 하지만 규정 문서에서는 질문과 문서 어휘가 거의 같아 가상 답변(HyDE)이나 질문 확장(Multi-Query)의 이득이 작고, 청크도 이미 짧아서 Compression이 얻을 게 별로 없었습니다. 비용은 올라가는데 숫자는 제자리였던 거죠."

체크한 일곱 항목이 커넥트HR 에이전트에 실제로 적용한 D 조합입니다. RAG 평가는 조합 밖에서 이 일곱 가지가 제값을 하는지 수치로 검증하는 장치로 따로 돌립니다.

## 10.7 전체 구성도에서 챕터 10의 자리

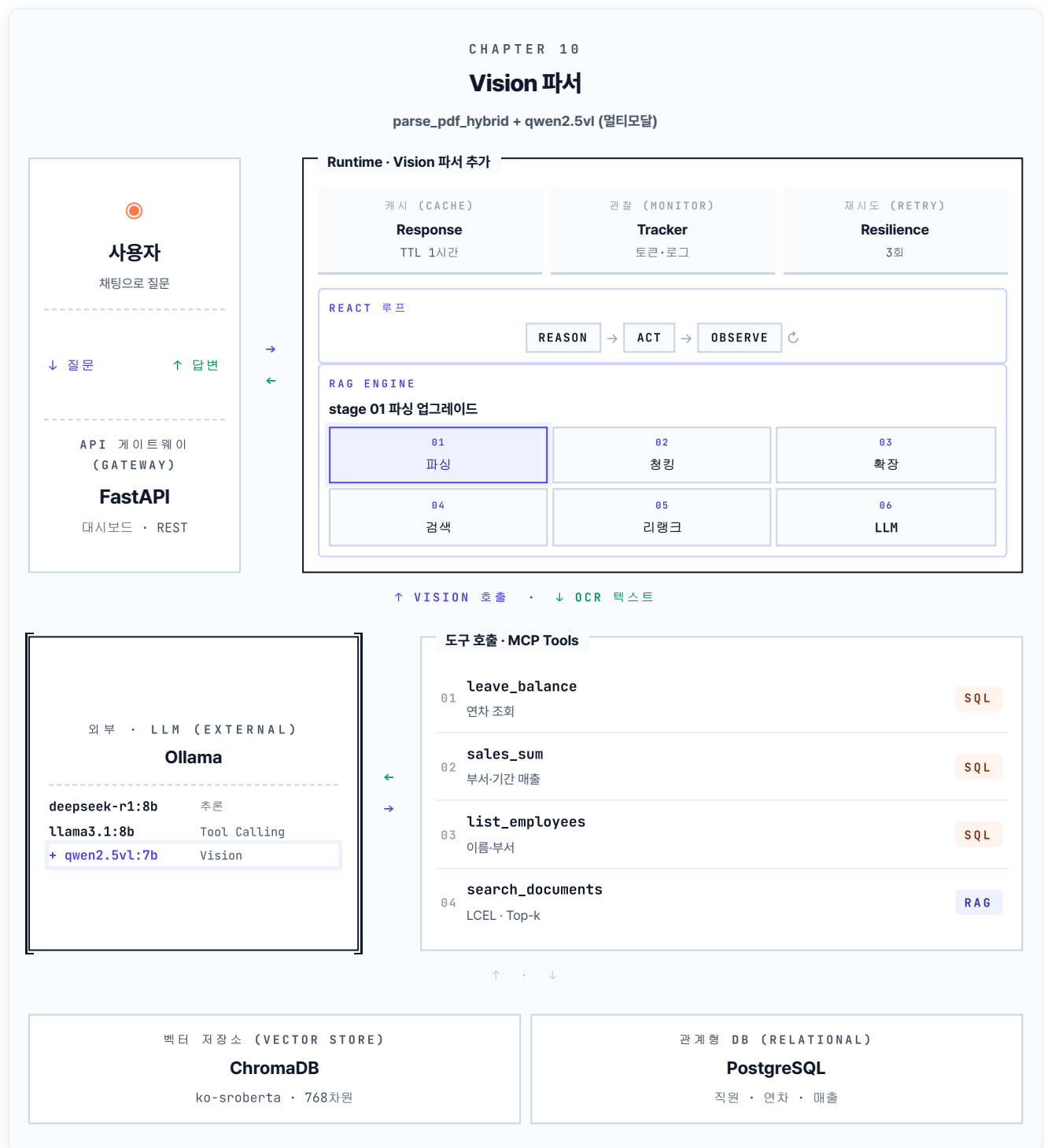
---

이번 챕터에서 RAG Engine의 첫 번째 stage(01 파싱)가 새로 손질됐습니다. 텍스트 레이어가 살아 있는 페이지는 그대로 pypdf가 처리하고, 스캔본처럼 텍스트가 비어 있는 페이지는 `parse_pdf_hybrid`가 Vision LLM에게 넘겨 글자를 꺼냅니다. 외부 LLM 자리에는 `qwen2.5vl:7b`가 새로 들어와, 추론용 deepseek-r1·도구 호출용 llama3.1과 함께 세 모델이 각자의 역할을 맡습니다.

여기에 챕터 8의 stage 05(리랭크)와 챕터 9의 stage 03(확장)이 누적돼 있어, 이제 RAG Engine 6단계 중 세 자리가 튜닝 작업으로 채워졌습니다. 청킹·하이브리드 검색·약어 확장·Parent Doc·리랭커·Vision 파서까지, 챕터마다 더한 부품이 한 그림 안에서 어떻게 자리를 잡았는지 한눈에 보이는 구성도입니다.

평가 프레임워크는 이 그림 안에 들어가지 않습니다. 파이프라인 옆에서 채점기 역할로 따로 돌면서, 다음 챕터에서 D 조합을 하나의 파이프라인으로 조립할 때 쓸 근거를 만들어 줍니다.

CH 10 · PDF 이미지까지 - stage 01 + qwen-vl NEW



새로: stage 01 파싱 (Vision) + qwen2.5vl 모델 · 다른 stage는 CH08(05 리랭크)·CH09(03 확장)에서 누적  
그림 10-13. 챕터 10의 결과. RAG Engine의 stage 01(파싱)이 업그레이드되고 qwen2.5vl 모델이 새로 자리를 잡았습니다

## 용어 정리

본문 속 표현	진짜 용어	정식 정의
"확대경으로 글자 읽기"	<b>OCR (Optical Character Recognition)</b>	이미지에서 문자를 인식해 텍스트로 변환
"눈을 가진 LLM"	<b>Vision LLM</b>	이미지·표·도식을 이해해 자연어로 설명하는 멀티모달 LLM
"이미지 감지 후 분기"	<b>하이브리드 파서</b>	<code>page.get_images()</code> · 텍스트 길이로 Vision LLM과 pypdf를 자동 선택
"정답 비율"	<b>Precision@k</b>	상위 k개 중 정답 청크 비율
"정답 커버리지"	<b>Recall@k</b>	정답 문서 중 상위 k개에 포함된 비율
"근거 없는 답변 비율"	<b>Hallucination Rate</b>	답변 문장 중 문서 근거가 없는 비율. 답변과 근거 문서의 단어 겹침으로 간단히 잡거나, 더 정확히 썰 땀 별도 LLM에 채점을 맡깁니다
"성적표"	<b>RAG Evaluation Framework</b>	평가셋(질문·정답)으로 파이프라인을 돌려 Precision·Recall·환각률을 수치화
"튜닝 조합 스위치"	<b>Strategy Pattern (A/B/C/D)</b>	파서·청킹·쿼리변환·검색·리랭크 부품을 갈아끼워 조합별로 성능을 비교
"응답 시간"	<b>Latency</b>	사용자 질문이 들어와서 답변이 나올 때까지 걸리는 총 시간. RAG에서는 검색 + LLM 호출 시간의 합



## 이것만은 기억하자

- **스캔 PDF는 pypdf로 안 읽힙니다.** 텍스트 레이어가 있는 페이지는 pypdf가 살리고, 그렇지 않은 페이지는 Vision LLM이 눈 역할을 합니다. 하이브리드 파서의 분기 기준은 **페이지에서 꺼낸 텍스트 길이**입니다.
- **RAG 평가는 네 지표 세트**입니다. Precision@k(정확성) · Recall@k(커버리지) · Hallucination Rate(근거 충실성) · Latency(속도). 한 면만 보면 다른 쪽이 무너집니다.
- **튜닝은 숫자로 증명된 것만 있습니다.** 우리 문서(규정·짧은 조항·약어 많음·스캔본 일부)에는 D 조합 (Semantic 청킹 + 하이브리드 검색 + 리랭커 + 약어 확장 + Parent Doc + Vision 파서)이 환각률과 커버리지 모두에서 가장 좋은 수치를 냈습니다. 스캔본이 없는 환경이라면 Vision 파서를 빼고 C로도 충분합니다. RAG 평가 프레임워크는 이 조합을 고를 때 쓴 채점기이지 파이프라인에 얹는 부품은 아닙니다.

다음 챕터에서는 이 D 조합을 **하나의 파이프라인으로 조립**하고, 답변에 **근거를 붙여** 사용자에게 내보냅니다. 커넥트HR 에이전트가 완성되는 마지막 장입니다.